



Chat frontend

The React application that assembles reusable features into a coherent workspace.

React SPA

Application adapters

Port 4207

apps/chat · @epam/chat



The frontend owns the user workflow

01

Conversation workspace

Send messages, resume history and inspect sources.

02

Knowledge workflows

Browse and edit prompts, skills, apps and toolsets.

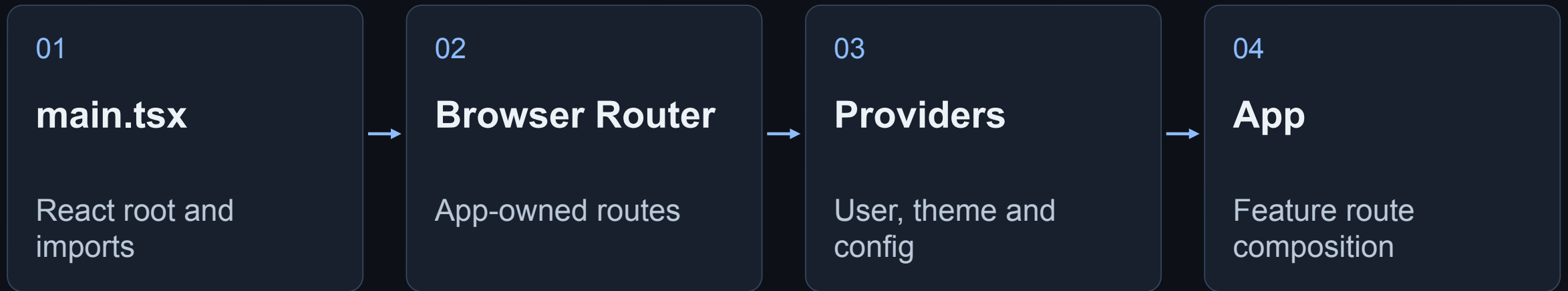
03

Operations for users

Inspect usage, manage files and work with scheduled tasks.



Bootstrap the application once



AttachmentCanvasProvider sits above the routed workspace.



Providers define state ownership

01

Identity and configuration

UserProvider,
AppConfigProvider and
UiFeaturesProvider.

02

Conversation state

ConversationsProvider,
GenerationProvider and
ClientChannelProvider.

03

Workspace presentation

Theme, sources-sidebar
and attachment-canvas
providers.

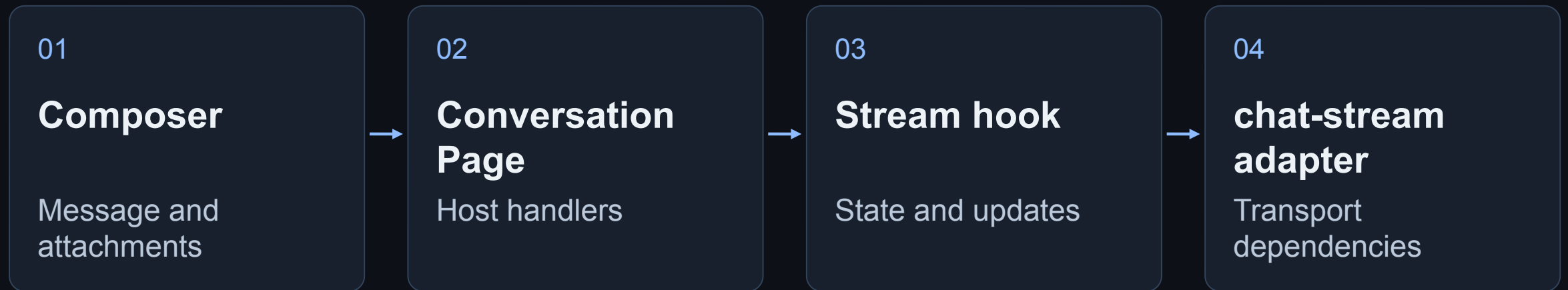


Routes connect reusable screens

Route family	Host responsibility
/conversations	Active conversation and history
/catalog, /files	Discovery and file operations
/prompt-editor, /skill-editor	Load, validate and save authored entities
/scheduled-tasks, /settings	Scheduling and usage/settings views



Trace a message from UI intent



The backend owns persistent conversation writes.



Configure the generated client at the edge

Source excerpt · `createApiConfiguration` in `server-api/api-client.ts`.

```
new Configuration({
  basePath: '',
  credentials: 'include',
  middleware: [
    csrfMiddleware,
    unauthorizedMiddleware,
    telemetryMiddleware,
  ],
});
```



Assemble evidence around each message

01

Transcript

The app message container composes bubbles and stages.

02

Sources

The app sources container adapts evidence data and open actions.

03

Canvas

The app resolves files and passes renderer dependencies.



Theme, locale and viewport are host concerns

01

ThemeProvider

Loads theme configuration and writes CSS variables.

02

i18n initialization

Registers English and updates html direction on language changes.

03

Responsive branches

App hooks provide mobile/desktop decisions to feature surfaces.



A concrete extension: message content slots

Runnable composition pattern from [examples/conversation-messages.tsx](#).

```
<AssistantMessageBubble
  text="**Demo:** evidence collected."
  isStreaming={false}
  deploymentDisplayName="Demo assistant"
  afterContent={
    <StagesPanel stages={[]} isStreaming={false} />
  }
/>
```



Inspect the reusable pieces in isolation

Reproducible demo · actual workspace source components

 DEMO · ACTUAL WORKSPACE COMPONENTS

Inspect an assistant workspace

Synthetic data · local source snapshot · no backend calls

Settings

- Usage
- Profile

Today

\$3.00 used of \$10.00

\$7.00 left 30%

WITHIN LIMITS

Response stages

☐ Read demo sources >

Retrieving evidence for the demonstration.

A focused demo

Use a small host to inspect component contracts before wiring application state and backend behavior.

DEMO · real components rendered from this source snapshot



Run and build the frontend

Commands run from the repository root.

```
npm start  
# Vite: http://localhost:4207  
# /api is proxied to http://localhost:5000
```

```
npm exec nx run @epam/chat:build  
npm exec nx run @epam/chat:typecheck  
npm exec nx run @epam/chat:test
```



Inspect behavior through existing tests

Test area	Evidence
Conversation route	ConversationRoute integration and unit specs
Message composition	ConversationMessageItem specs
Provider behavior	OverlayContext and app-level context specs
API adapters	server-api tests for transport integration



Integration limits are part of the design

01

Providers are coupled

The full app composition has many coordinated state owners.

02

Backend required

A running UI alone cannot authenticate or complete conversations.

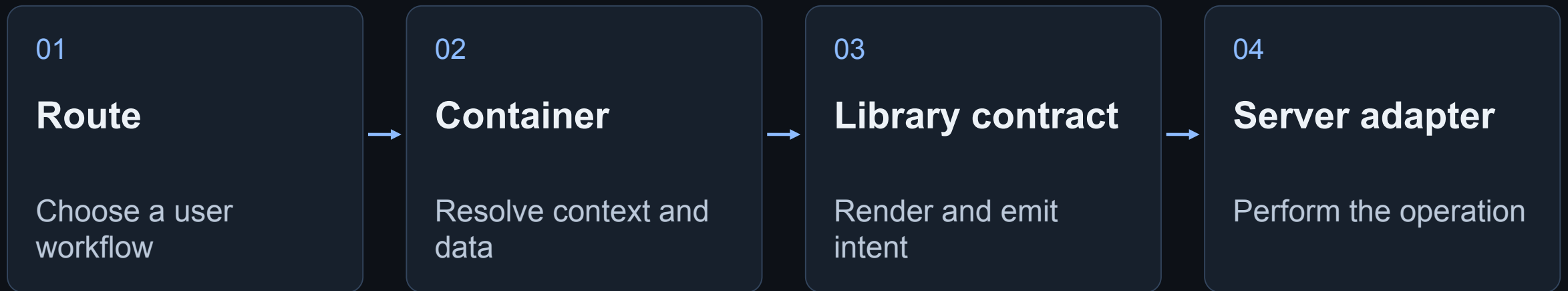
03

Some adapter gaps remain

Audio transcription retains a documented raw DTO wrapper.



Follow one feature end to end



Start in `apps/chat/src/pages`, then follow imports into `libs` and `server-api`.